

Data Structure

Data structure is a systematic way to organise data in order to use it efficiently.

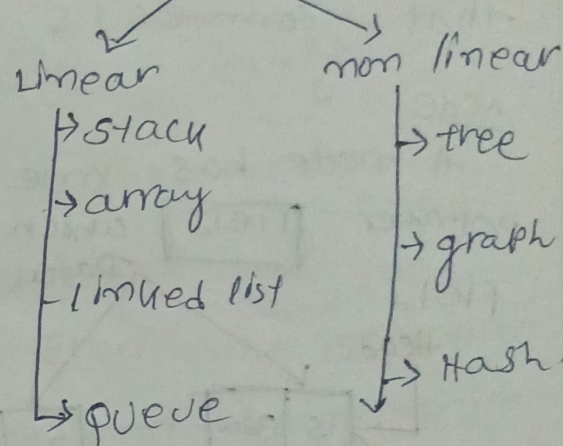
Data structure is collection of data value & the relationship among them and function or operation that can be applied to the data.

Data structure is generally classified into two types

Primitive data

- integer
- real number
- character
- Boolean

Non primitive data



Linear data structure

Data items that are arranged and accessed in a linear sequence are known as linear data structure. ex: array, linked list, stack etc.

Array:

An array is a linear data structure that stores multiple elements of the same data type in contiguous memory location and access by their index.

Syntax

data type array_name [size];

ex

int arr[5] = {0, 10, 20, 50, 30}

memory →

	1001	1003	1005	1007	1009
	0	10	20	50	30

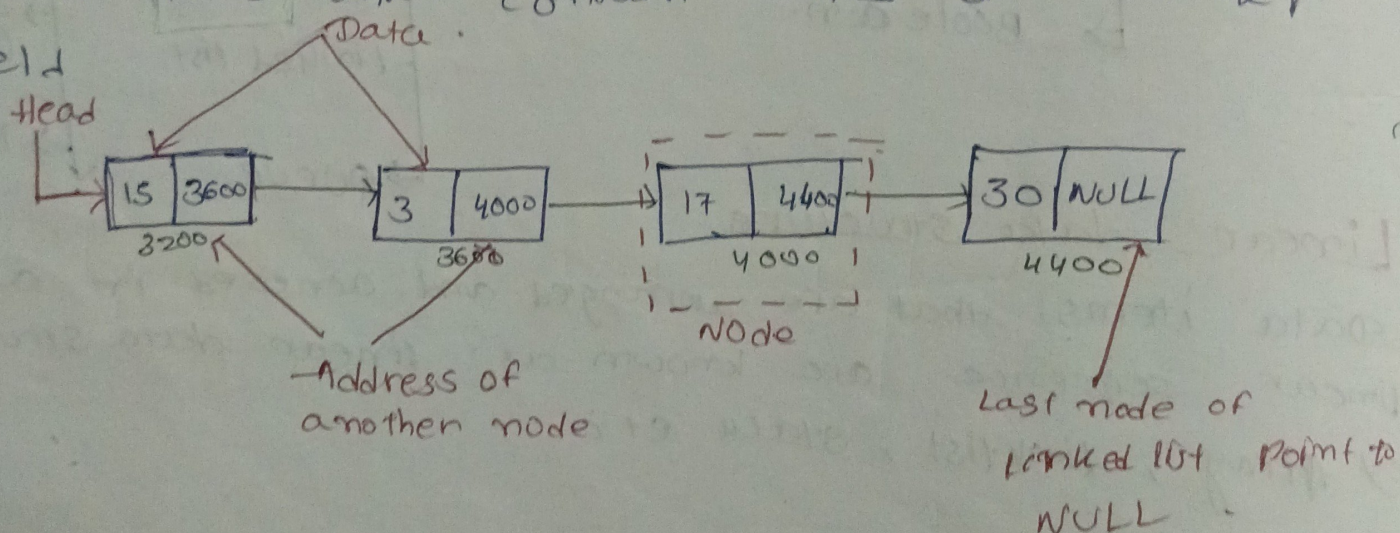
index → 0 1 2 3 4

Linked List

A linked list is a linear data structure made up of nodes where each node contains data and a pointer that connects data and a pointer that connects it to next node in the list.

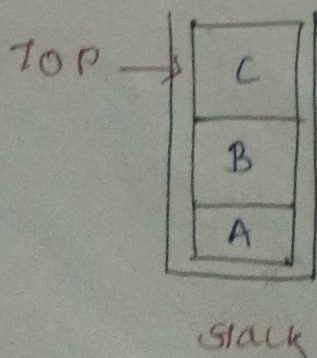
Node

A node has one information field and one pointer field which contain address of another field.



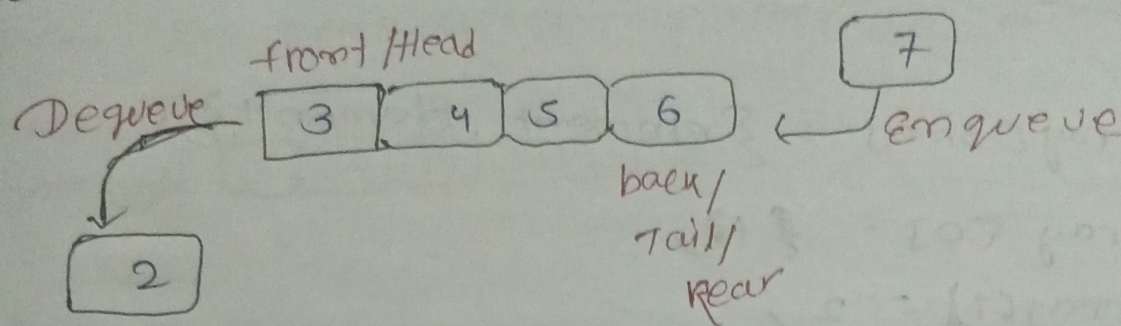
Stack

A stack is a Last in first out system in which insertion and deletion takes place at one end i.e. at top end.



Queue

Queue is a first in first out (FIFO) system in which deletion takes place at one end and insertion takes place at other end.



Data Structure operation

1. Traversing $\rightarrow$ Accessing each record exactly.
2. Searching $\rightarrow$ finding the location of the record.
3. Inserting $\rightarrow$ Adding a new element.
4. Deleting $\rightarrow$ Remove a record from the structure.
5. Sorting $\rightarrow$ Arranging the records in logical manner.
6. Merging $\rightarrow$ connecting the records.

Array:

An array is collection of similar elements of same data type that are stored in a contiguous memory location.

- Array elements are accessed by their reference index value.

memory representation

All element in an array ~~have~~ are stored in a contiguous memory location.

Declaration of Array

Datatype: `array_name[size];`

Initialization

way-

1) `int array[size] = {element, element...}`

2) `int array[0] = 1;`

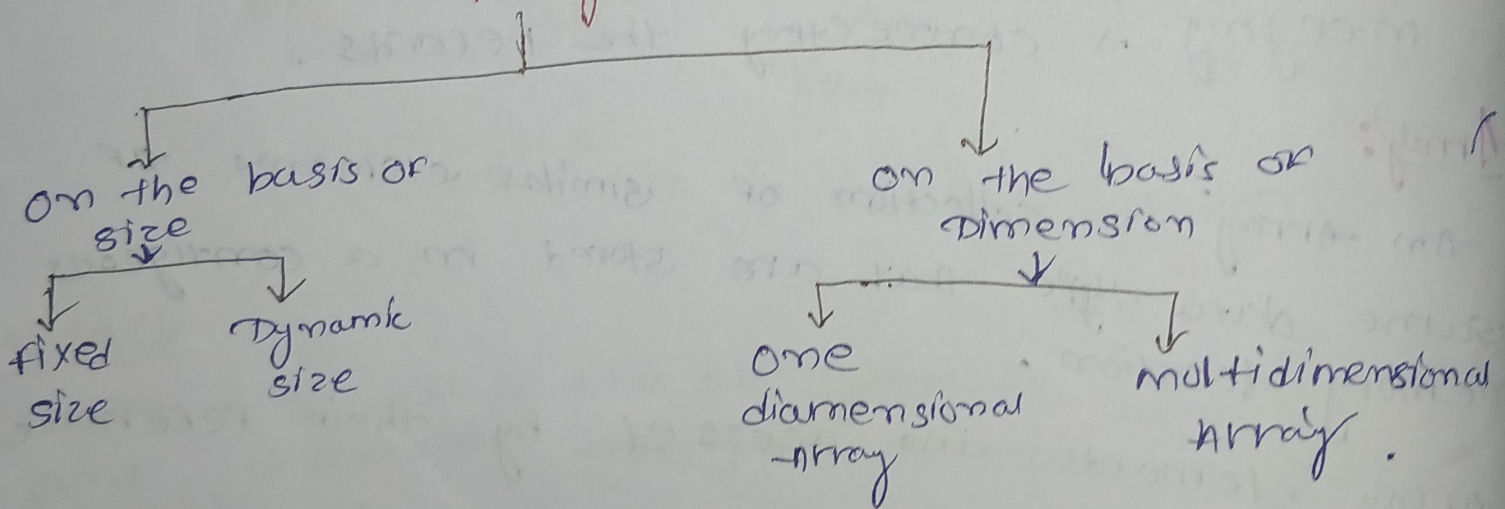
`array[1] = 2;`

`array[2] = 3;`

Types of Array

- Array can be classified into two types
- On the basis of size
- On basis of dimension.

Array



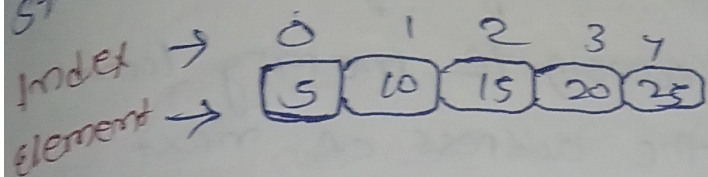
i) On the basis of size :-

a) fixed size.

In array they have fixed size. We cannot update the size of array.

1. 1 Dimensional Array

In one dimensional array, array elements are stored in one after another in a row.



2. MULTIdimensional Array

A multidimensional array is an array with more than one dimension.

We can use multidimensional array to store complex data in the form of tables. We can have 2-D array, 3-D array, 4-D array and so on.

i) 2-D Array

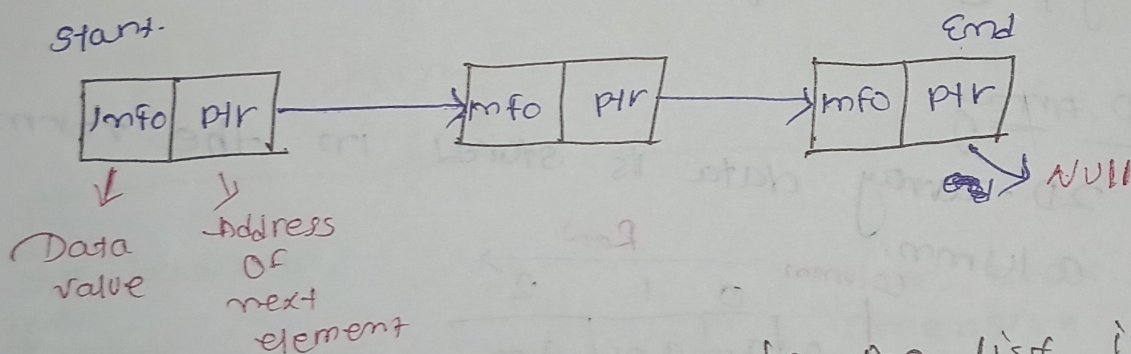
In 2D-array data is stored in the form of Row and column.

	Row		
Column	0	1	2
0	a_{00}	a_{01}	a_{02}
1	a_{10}	a_{11}	a_{12}
2	a_{20}	a_{21}	a_{22}

3x3

Linked List

- Linked list uses the concept of dynamic memory allocation.
- It is useful because the exact storage space can be determined at the time of beginning.
- Since linked list are not contiguous memory location, each item must contain the address of next item.
- The linked list is a unit or structure which contains two fields one is informational field and another is pointer field.
- The entire list is accessed from an external pointer 'start', which points to the first element and the last element's pointer field contains a null value which indicates end of the linked list.



- When there is no element in the list it is known as an empty list or null list and the start points to null.

Basic step in dealing with linked list

- Declare the structure that defines the list

```
struct node {  
    data member  
    ~ ~ ~  
};
```

Declare the "start" and initialize it to null.

```
struct node *start = NULL
```


for creating a node use malloc function and assign to a pointer.

struct node * new node;

new node = (struct node *) malloc (size of (struct node));

assign value to new node's information field.

assign address of the new node pointer.

```
struct node {  
    int data;  
    struct node * next;  
}
```

~~void main()~~

~~struct node * start = NULL;~~

~~struct node * new node;~~

new node = (struct node *) malloc (size of (struct node));

new node → data = 5;

new node → next = NULL

start = new node.

struct * new node 1;

new node 1 = (struct node *) malloc (size of (struct node));

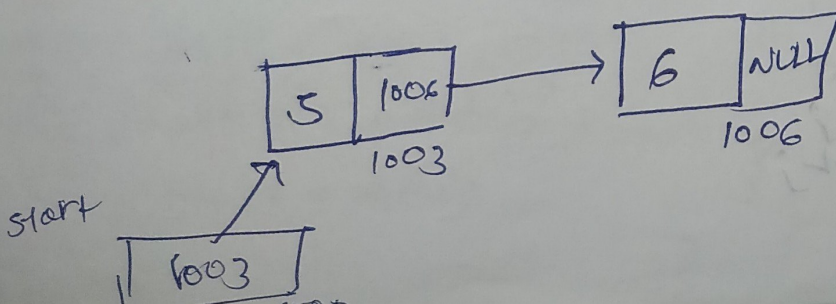
new node 1 → data = 6;

new node 1 → next = NULL

~~new node~~
start → next = new node 1;

}

Visualization



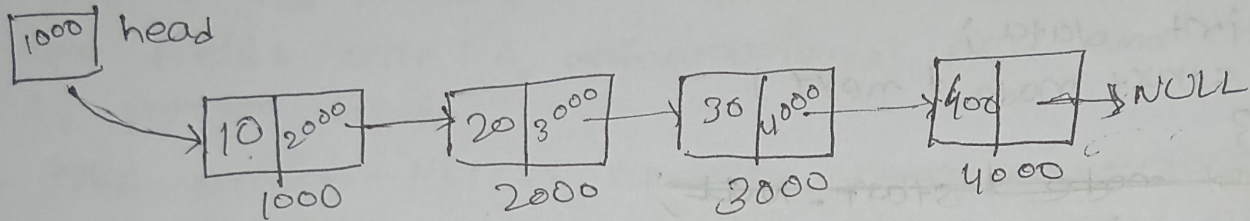
Single linked list

Types of linked list

1. Single linked list
2. Doubly linked list
3. Circular linked list.

Single linked list

- A singly linked list is a linear data structure where elements (nodes) are connected using pointers.



creating a single linked list
#with out using function

code

```
#include <stdio.h>
#include <stdlib.h>
struct node {
```

```
    int data;
```

```
    struct node *next;
};
```

```
void main ( ) {
```

```
    struct node *new, *new1, *new2, *start = NULL;
```

```
    new = (struct node *) malloc (sizeof (struct node));
```

```
    new1 = (struct node *) malloc (sizeof (struct node));
```

```
    new2 = (struct node *) malloc (sizeof (struct node));
```

```
    new->data = 5;
```

```
    new1->data = 500;
```

```
    new2->data = 5000;
```

```
    new->next = NULL;
```

```
    start = new;
```



```

new1 → next = new2;
new2 → next = NULL;
new1 → next = new2;

```

```

struct node *ptr;
ptr = start;

```

```

while (ptr != NULL) {

```

```

    ptr → next;

```

```

    printf("%d \n", ptr → data);

```

```

    ptr = ptr → next;

```

```

}

```

```

}
OUTPUT
5
500
500 0

```

// using function

```

#include <stdio.h>

```

```

#include <stdlib.h>

```

```

struct node {

```

```

    int data;

```

```

    struct node *next;

```

```

}

```

```

struct node *start = NULL;

```

```

void create(int value) {

```

```

    struct node *new;

```

```

    new = (struct node *) malloc
        (sizeof(struct node));

```

```

    new → data = value;

```

```

    new → next = NULL;

```

```

    if (start == NULL) {

```

```

        start = new;

```

```

    }

```

```

    else {

```

```

        struct node *ptr = start;

```

```

        while (ptr → next != NULL) {

```

```

            ptr = ptr → next;

```

```

        }

```

```

        ptr → next = new;

```

```

    }

```

```

void display() {

```

```

    struct node *ptr = start;

```

```

    while (ptr != NULL) {

```

```

        printf("%d \n", ptr → data);

```

```

        ptr = ptr → next;

```

```

    }

```

```

}

```



```

int main() {
    create CS);
    create C10);
    create C15);
    display C);
    return 0;
}

```

OUTPUT:-

5
10
15

Operations on singly linked list

1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Updating
6. Reversal.

1. Traversal of singly linked list

- Traversal of singly linked list is one of the fundamental operation. where we traverse each node of the linked list.
- The process of traversing a singly linked list involves printing the value of each node and then going on to next node and print that node's value also and so on.

Algorithm

- we define or initialize a temporary pointer to the head node of the singly linked list.
- After that we will check if that pointer is null or not null, if it is null then return

• while the pointer is not NULL, we will access the and print the data of the current node, then, we move the pointer to next node.

code

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
    int data;
    struct node *next;
};
```

```
;
```

```
struct node *start = NULL;
```

```
int main () {
```

```
struct node *new, *new1, *new2, *new3;
```

```
struct node *ptr = start
```

```
new = (struct node *) malloc (sizeof (struct node));
```

```
new1 = (struct node *) malloc (sizeof (struct node));
```

```
new2 = (struct node *) malloc (sizeof (struct node));
```

```
new3 = (struct node *) malloc (sizeof (struct node));
```

```
new → data = 500;
```

```
new1 → data = 27;
```

```
new2 → data = 04;
```

```
new3 → data = 2008;
```

```
new → next = NULL;
```

```
start = new;
```

```
new1 → next = NULL;
```

```
new → next = new1;
```

```
new2 → next = NULL;
```

```
new1 → next = new2;
```

```
new3 → next = NULL;
```

```
new2 → next = new3;
```

```
// traverse
```

```
while (ptr != NULL)
```

```
struct node *ptr = start;
```



```

while (ptr != NULL) {
    printf(" %d \n", ptr->data);
    ptr = ptr->next;
}
}

```

OUTPUT:-

```

500
27
04
2008.

```

- write a programme to create and display a linked list using function?

CODING:-

```

#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};

```

```

struct node *start = NULL;

```

```

void create (int data) {

```

```

    struct node *new;

```

```

    new = (struct node *) malloc (sizeof (struct node));

```

```

    new->data = data;

```

```

    new->next = NULL;

```

```

    if (start == NULL) {

```

```

        start = new;

```

```

    }

```

```

    else {

```

```

        struct node *b = start;

```

```

        while (b->next != NULL) {

```

```

            b = b->next;

```

```

        }

```


b → next = new;

```
}  
void display C {  
    struct node *ptr = start;  
    while (ptr != NULL) {
```

```
        ptr = ptr->next  
        printf("%d\n", ptr->data);  
        ptr = ptr->next;  
    }
```

```
}  
void main C {  
    create C20;
```

```
    create C25;
```

```
    create C40;
```

```
    display C;
```

```
}
```

Output

20
25
40.

Operation on singly linked list:

1. Traversal:

Traversal means visiting each node one by one from the head to the last node to access or display data.

Algorithm:

1. start

2. Set temp = head

3. Repeat while temp ≠ NULL

- Display temp → data
- move to next node

4. stop


```

#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node * next;
};

void main() {
    struct node * start = NULL;
    struct node * new1;
    struct node * new2;
    struct node * new3;
    new1 = (struct node *) malloc (sizeof
                                   (struct node));
    new2 = (struct node *) malloc (sizeof
                                   (struct node));
    new3 = (struct node *) malloc (sizeof
                                   (struct node));

    new1->data = 23;
    new1->next = NULL;
    start = new1;
    new2->data = 24;
    new2->next = NULL;
    start->next = new2;
    new3->data = 25;
    new3->next = NULL;
    new2->next = new3;

    // traversal
    while struct node * ptr = start;
    while (ptr != NULL) {
        printf("%d\n", ptr->data);
        ptr = ptr->next;
    }
}

```

23
24
25

• using function.

```

#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node * next;
};

struct node * start = NULL;
void create (int data) {
    struct node * new;
    new = (struct node *) malloc (
        sizeof (struct node));

    new->data = data;
    new->next = NULL;
    if (start == NULL) {
        start = new;
    }
    else {
        struct node * ptr = start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = new;
    }
}

```


// traversal

```
void traverse ( ) {
    struct tnode * ptr = NULL;
    ptr = start;
    while ( ptr != NULL ) {
        printf("%d\n", ptr->data);
        ptr = ptr->next;
    }
}
```

OUT PUT:

40
50
60

```
}
void main ( ) {
    create (40);
    create (50);
    create (60);
    traversal ();
}
```

2. INSERTATION :

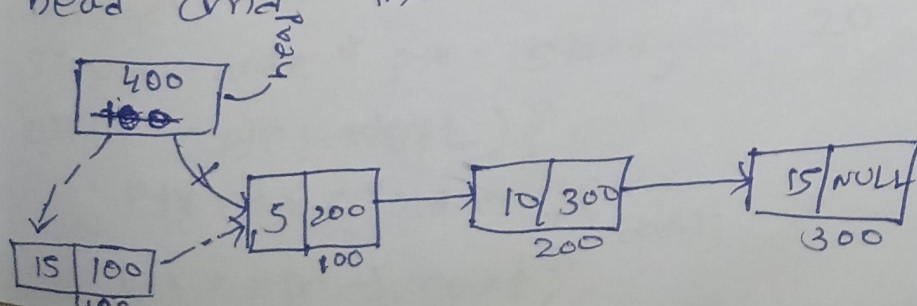
Insertion in a singly linked list involves adding a new node at a specified position in the list -

- There are several types of insertion based on the position:-

1. At the beginning
2. Insertion in a specific position
3. At the end.

1. At the beginning

To insert a new node at the front, we create a new node and point its "next" to the head and then update head equal to new node.



without using function.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
}
```

```
void main()
```

```
{  
    struct node *new1, *new2, *new3;
```

```
    struct node *start = NULL;
```

```
    new1 = (struct node *) malloc (
```

```
        sizeof(struct node));
```

```
    new2 = (struct node *) malloc (sizeof
```

```
        (struct node));
```

```
    new3 = (struct node *) malloc (sizeof
```

```
        (struct node));
```

```
    new1->data = 24;
```

```
    new1->next = NULL;
```

```
    start = new1;
```

```
    new2->data = 25;
```

```
    new2->next = NULL;
```

```
    new1->next = new2;
```

```
    new3->data = 26;
```

```
    new3->next = NULL;
```

```
    new2->next = new3;
```

```
    struct node *node;
```

```
    node->data = 24;
```

```
    node->next = head;
```

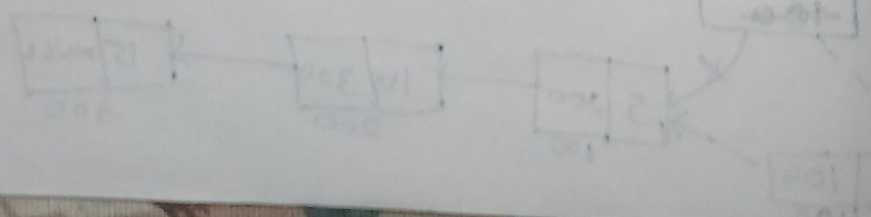
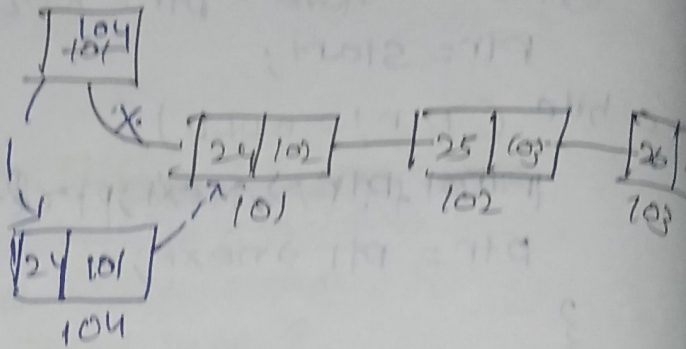
```
    start = node;
```

```
}
```

```
}
```

OUTPUT

Structure




```

using namespace std;
#include <stdio.h>
#include <stdlib.h>

```

node

```

struct node {
    int data;
    struct node * next;
};

```

```

struct node start = NULL;
void create (int data) {
    struct node * new;
    new = (struct node *) malloc
        (sizeof (struct node));

```

```

    new->data = data;

```

```

    new->next = NULL;

```

```

    if (start == NULL) {

```

```

        start = new;
    }

```

```

    else {

```

```

        struct node * ptr = start;

```

```

        while (ptr->next != NULL) {

```

```

            ptr = ptr->next;

```

```

        }

```

```

        ptr->next = new;
    }

```

```

}

```

```

void display () {

```

```

    while (

```

```

        struct node * ptr = start;

```

```

        while (ptr != NULL) {

```

```

            printf ("%d\n", ptr->data);

```

```

            ptr = ptr->next;
        }
    }

```

```

}
void insert at begin (int data) {

```

```

    struct node * new;

```

```

    new = (struct node *) malloc (
        sizeof (struct node));

```

```

    new->data = data;

```

```

    new->next = NULL;

```

```

    new->next = start;

```

```

    start = new;
}

```

```

void main () {

```

```

    create (5);

```

```

    create (10);

```

```

    create (15);

```

```

    create (20);

```

```

    insert (25);

```

```

    insert at beginning (25);

```

```

    display ();
}

```

OUTPUT:-

25

5

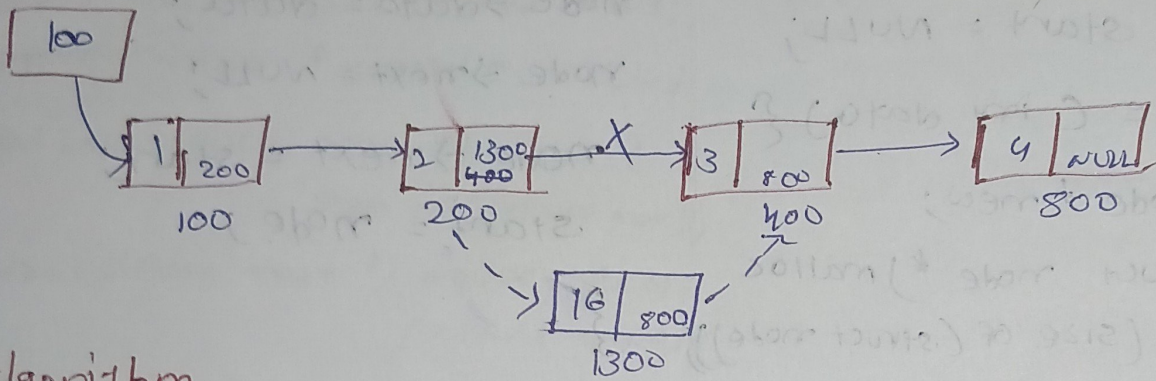
10

15

20

1) Insertion at any position.

To insert a node at a specific position we need to traverse the list to position - 1. If the position is valid, we adjust the pointer similarly such that the next pointer of new node points to the next of current node and next pointer of current node points to the new node.



Algorithm.

1. create new node

- allocate memory for new node
- set new node $\rightarrow$ data = data
- set new node $\rightarrow$ next = null.

2. check if position is 1 (beginning)

- If $pos == 1$
 - new node $\rightarrow$ next = head
 - head = new node
 - return head.

3. Traverse to (pos-1) node

- initialize temp = head
- Repeat (pos-2) times:
 - move temp = temp $\rightarrow$ next
 - If temp = NULL $\rightarrow$ position invalid & stop

4. Insert
 • new node $\rightarrow$ next = temp $\rightarrow$ next
 • temp $\rightarrow$ next = new node.

5. Return updated head.

CODING:

```
#include <stdio.h>
#include <stdlib.h>

void create (int data) {
    struct node {
        int data;
        struct node * next;
    } * start;

    void create (int data) {
        struct node * new;
        new = (struct node *) malloc (
            sizeof (struct node));

        new->data = data;
        new->next = NULL;
        if (start == NULL) {
            start = new;
        }
        else {
            struct node * ptr = start;
            while (ptr->next != NULL) {
                ptr = ptr->next;
            }
            ptr->next = new;
        }
    }
}
```

```
void display () {
    struct node * ptr = start;
    while (ptr != NULL) {
        ptr = ptr->next;
    }
}
```

```
}
}
void insert at any (int data,
int pos) {
    int i;
    struct node * new;
    new = (struct node *) malloc
        (sizeof (struct node));
```

```
new->data = data;
new->next = NULL;
for (i = 1; i < pos; i++) {
    struct node * ptr = start;
    for (i = 1; i < pos; i++) {
        ptr = ptr->next;
    }
}
```

```
ptr->next = new->next;
new->next = ptr->next;
ptr->next = new;
}
```

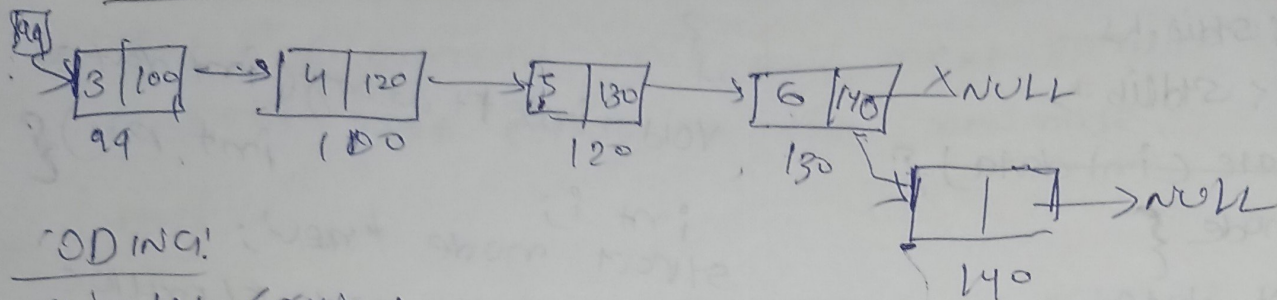
```
void main () {
```

```
    create (25);
    create (15);
    create (10);
    insert at any (20, 2);
    display ();
}
```


OUTPUT

25
20
15
10

3) Insertion at end



CODEING:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void cr  
struct node {
```

```
int data;
```

```
struct node * next;
```

```
};
```

```
struct node * start = NULL;
```

```
struct
```

```
void create (int data) {
```

```
struct node * new;
```

```
new = (struct node *) malloc(  
sizeof (struct node));
```

```
new->data = data;
```

```
new->next = NULL;
```

create
display

```
void insert at end (int data)
```

```
struct node * new;
```

```
new = (struct node *) malloc(  
sizeof (struct node));
```

```
new->data = data;
```

```
new->next = NULL;
```

```
struct node * ptr = NULL;
```

```
ptr = start;
```

```
while (ptr->next != NULL) {
```

```
ptr = ptr->next;
```

```
}
```

```
ptr->next = new;
```

```
}
```

```
void main () {
```

```
create (25);
```

```
create (20);
```

```
insert at end (28);
```

```
display ();
```

OUTPUT

25
20
28

Linked List

Deleting a node in a linked list is an operation and can be done in three main ways:

1. Deletion at beginning.
2. Deletion at specific position.
3. Deletion at end.

1. Deletion at beginning

Deletion at the beginning involves removing the 1st node of the linked list.

CODE

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};
```

```
struct node *start = NULL;
```

// creation

```
void create(int data) {
```

```
    struct node *new;
```

```
    new = (struct node *) malloc (sizeof struct node);
```

```
    new->data = data;
```

```
    new->next = NULL;
```

```
    if (start == NULL) {
```

```
        start = new;
```

```
    }
    else {
```

```
        struct node *p = start;
```

```
        while (p->next != NULL) {
```

```
            p = p->next;
```

```
        }
```

```
        p->next = new;
```

```
    }
```



```
void delete ( ) {
```

```
    struct node * temp = NULL;
```

```
    temp = start; next
```

```
    start = next = temp = next;
```

```
    free (temp);
```

```
}
```

```
void display ( ) {
```

```
    struct node * ptr = start;
```

```
    while (ptr != NULL) {
```

```
        ptr = ptr->next;
```

```
    }
```

```
int main ( ) {
```

```
    create (5);
```

```
    create (10);
```

```
    create (15);
```

```
    create (20);
```

```
    delete ( );
```

```
    display ( );
```

```
}
```

OUTPUT

10

15

20

2. Deletion at a specific position in a linked list

Deletion at a specific position involves removing the node at a specific position in a linked list.

code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node * next;
```

```
};
```



```

struct node *create (int data, struct node *head) {
    struct node *new;
    new = (struct node *) malloc (sizeof (struct node));
    new → data;
    new → next;
    if (head == NULL) {
        head = new;
    } else {
        struct node *ptr = head;
        while (ptr → next != NULL) {
            ptr = ptr → next;
        }
        ptr → next = new;
    }
    return head;
}

```

```

struct node *delete_at_pos (int pos, struct node *head) {
    struct node *ptr = NULL;
    struct node *temp = NULL;
    ptr = head;
    while (pos > 1; ptr = ptr → next) {
        ptr = ptr → next;
    }
    temp = ptr → next;
    ptr → next = temp → next;
    free (temp);
    return head;
}

```

```

struct node *display (int head) {
    struct node *ptr = head;
    while (ptr != NULL) {
        printf ("%d\n", ptr → data);
        ptr = ptr → next;
    }
}

```



```
int main() {
```

```
    struct node *head = NULL;
```

```
    head = create(5);
```

```
    head = create(10);
```

```
    head = create(15);
```

```
    head = deleteany(2);
```

```
    display(head);
```

output

5
15

Double linked list

A doubly linked list is a type of linked list in which each node contains 3 parts, a data part and two addresses, one points the previous node and one for the next node.

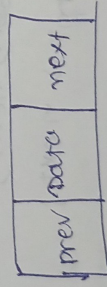
- Each node in doubly linked list contains

three components: data, next, prev.



Implementation of doubly linked list

To implement a doubly linked list first we need to define a node that has three parts: data, a pointer to next node, and a pointer to previous node.



```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
    struct node *prev;
```


Operation on doubly linked list.

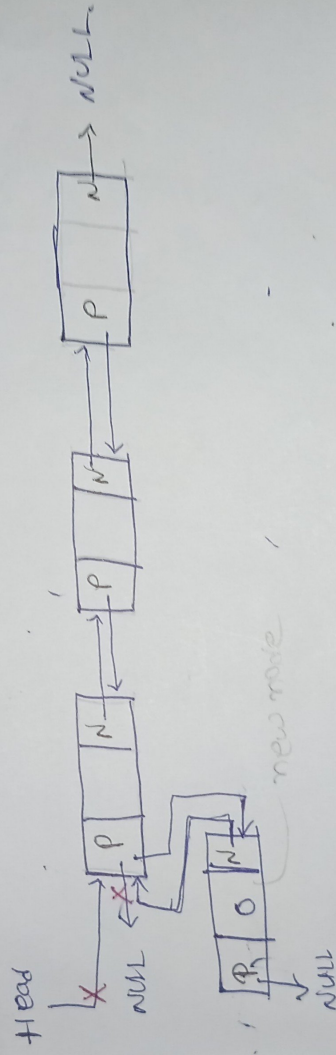
- Insertion
- Deletion
- Traversal.

1. Insertion

Inserting a new node in a doubly linked list & to maintaining the link to the previous and next node is called insertion of linked list.

- There are three scenarios - while inserting a node in a doubly linked list.

- a) Insertion at the beginning
- b) Insertion at end
- c) Insertion at any position in between the linked list.



Algorithm

Step 1: Start

Step 2: Create a new node N.

Step 3: Store the data in N.

Step 4: Set $N \rightarrow \text{prev} = \text{NULL}$.

Step 5: If list is empty

i) set $N \rightarrow \text{next} = \text{NULL}$

ii) set head = N

Step 6: else

• set $N \rightarrow \text{next} = \text{head}$

• set head $\rightarrow \text{prev} = N$

- set head = N

step 6: stop.

Time complexity: O(1)

Space complexity: O(1)

b) Insertion at the end of Doubly linked list.

- Inserting a new node at the end of doubly linked list and maintain linked between them is called as insertion at the end.

